Integration Layer Design: 3D Sales Workspace

Backend Assessment, Part A (architecture and design) · Candidate: Rohit Sharma · 20 October 2026

1. Architecture flow

1.1 Trust zones

Zone	Contains	Trust
Untrusted zone	Browser (React workspace with the 3D room and sidebar)	Untrusted. Every request is authenticated and authorised on the server.
Customer identity	OpenID Connect identity provider	Issues and signs tokens; the API verifies them against its JWKS.
Customer Azure tenant	Firewall (WAF), Integration API, Redis, PostgreSQL, workers, adapters, Blob storage, Key Vault, monitoring	Trusted. Our code and data stores.
Enterprise systems	Sizing tool, Pricing ERP, Catalogue, Document vault, Host project API	Owned by other teams and cannot be changed. Their responses are validated as untrusted input.

1.2 Read flow: selecting an item fills the sidebar

1. The user signs in through the identity provider (OIDC authorisation code with PKCE) and the browser receives an access token.
2. The browser calls the Integration API over HTTPS through the customer WAF, sending the token as a Bearer header.
3. The API verifies the token (signature against JWKS, issuer, audience, expiry) and builds the caller context: role, country and customer group.
4. The API loads the item from PostgreSQL and fans out to the adapters in parallel. Adapters run in-process, one per system.
5. Each adapter checks the Redis cache first. On a miss it takes a token from the global rate limit in Redis and calls its system over HTTPS REST.
6. Pricing is called only if the caller is entitled to see prices. The entitlement check runs before the cache is read.
7. Every vendor response is validated against a schema. The API returns one section per system, each with its own status, so one failing system never blanks the sidebar.

1.3 Write flow: re-sizing an item

1. The browser sends the re-size request with an idempotency key, so a retried click cannot apply twice.
2. The API calls the sizing tool through its adapter, with the user's delegated (OBO) token.
3. If the recommended variant changed, one PostgreSQL transaction updates the item's variant and 3D model reference, its BOM line and its version, and writes an outbox row.
4. A worker reads the outbox and saves the change to the Host project API with `If-Match` on the version, so a save is never duplicated and never overwrites a newer change.

1.4 Background and slow work

- **Job queue in Redis.** Slow calls and retries become jobs. The job ID is built from type, item, version and change, so duplicate requests merge into one job.
- **Slow-service worker.** Runs queued calls to slow or rate-limited systems through the same adapters.
- **Scheduled worker.** Imports the catalogue nightly export and change feed into PostgreSQL with an upsert by revision, so an old update never overwrites a newer one. It stores 3D models in Blob storage.
- **Dead-letter queue.** Jobs that keep failing go to the DLQ for inspection and alerting.

1.5 Inbound events from the host

The Host project API sends a webhook over HTTPS when a project changes elsewhere. It passes the WAF, its HMAC signature is verified, and its event ID is checked so a repeated event is processed only once.

1.6 Direct downloads

- **3D models:** the browser downloads glTF files from Blob storage with a SAS signed URL.
- **Documents:** the browser downloads a file directly from the Document vault with a short-lived signed URL.

Large files never pass through the Integration API.

1.7 Connections to each enterprise system

System	Protocol	Credential	Limits and rules

Sizing tool	HTTPS REST	User token (OBO)	Max 18 req/s across the whole service
Pricing ERP	HTTPS REST	Service account	Max 4.5 req/s; entitlement checked before every call
Catalogue	Nightly export and change feed	Service account	Read by the scheduled worker, not on the request path
Document vault	HTTPS REST	User token (OBO)	Per-document permissions
Host project API	HTTPS REST; webhooks back	User token (OBO)	`If-Match` version on save

1.8 Supporting services

- **Key Vault:** holds vendor credentials and the webhook key; no secrets in code.
- **Monitoring:** collects logs, metrics and traces from all services.

1.9 AI assistant flow

The assistant runs in its own container inside the tenant and never calls an enterprise system directly. Every tool it uses is a call to the same Integration API services the sidebar uses, made with the user's own context, so it can never see more than the user can.

1. **Question in.** The browser sends the question to the Integration API (HTTPS, Bearer JWT). The API verifies the token and forwards the question with the caller context to the Assistant container over internal HTTPS.
2. **Orchestrator.** Picks the module agent that fits the question (BOM, sizing or catalogue) and loads the conversation memory from PostgreSQL.
3. **Module agent.** Reasons with the Private LLM over a private endpoint. The LLM has no credentials and no outbound internet, so it can only suggest tool calls, never perform them.
4. **Read or write?** A read tool (for example `get_item_details`, `get_bom_cost`) runs straight away. A write tool (for example `resize_item`) goes to **human in the loop**: the user sees the proposed change and must confirm it. On confirm, the UI receives a signed, single-use confirmation bound to the user, item and arguments. The model cannot produce this confirmation itself. On decline, the agent is told no and nothing changes.
5. **Tool registry (MCP).** Tool calls carry only IDs and parameters. They run as the user, against the Integration API services, through the same entitlements, cache, adapters and rate limits as the sidebar.
6. **Validator / formatter.** Checks the draft answer before it leaves: no price if the user may not see prices, plain text only, and every figure must come from a tool result. If a check fails, the answer goes back to the agent with a critique.
7. **Answer out.** The answer is returned to the Integration API and streamed to the browser over SSE. The question, every tool call, entitlement decisions, confirmations and the answer are written to the audit log in PostgreSQL.

Prompt injection. Catalogue text and document content are untrusted. They reach the model only as clearly separated data, never as instructions. Even if injected text convinces the model, it still cannot exceed the user's permissions, cannot write without the user's confirmation, and has no network path to send data out.

1.10 Example: re-sizing a drive through the assistant

A sales engineer in Germany asks the assistant: "*Increase the motor on DRV-03 to 90 kW and re-size it.*" DRV-03 is currently variant `ACS-R7-75-400` (frame R7), at item version 7. The steps below follow the request through every component of the architecture.

#	Component	What happens
1	Browser → WAF → Integration API	The question is sent with the Bearer token. The API verifies the token and builds the caller context: `sales_engineer`, country DE, customer group G1.
2	Assistant container	The orchestrator routes the question to the sizing agent. With the Private LLM, the agent prepares a preview call: `preview_resize(DRV-03, 90 kW, 400 V)`. A preview only reads, so it needs no approval.
3	Integration API: sizing read order	① Redis `size:{hash(inputs, apiVersion)}`: miss. ② PostgreSQL sizing results for that input hash: miss. ③ Only now is the sizing adapter called.
4	Sizing adapter	Checks the circuit breaker (closed), takes a token from the sizing rate limit in Redis, and uses the cached OBO token for this user. A sizing run takes 1 to 6 s, so the call becomes an interactive job; its job ID is the input hash, so identical requests from other users merge into the same job.
5	Worker (interactive) → Sizing tool	The worker calls the sizing tool over HTTPS REST. The response is validated against its schema: recommended variant `ACS-R8-90-400` (frame R8), 88 % load. ④ The result is written to PostgreSQL and to Redis (fresh for 15 minutes).
6	Human in the loop	The agent shows the proposal: R7 → R8, the BOM change and the price difference. The user clicks **Confirm** and the UI receives a signed confirmation valid for 5 minutes.
7	Tool registry → Integration API	`resize_item` runs with the confirmation, the user's context and `If-Match: 7`. The sizing result is now a Redis hit, so the sizing tool is **not** called a second time.

8	PostgreSQL (one transaction)	The variant changed, so one transaction updates the item's variant and 3D model reference, its BOM line, sets version 8 and writes an outbox row. Either all of it is saved or none of it is.
9	Pricing	The entitlement check passes for a sales engineer. Price for R8 in DE/G1: Redis miss, then a valid PostgreSQL price snapshot, else the pricing adapter calls the ERP within its rate limit.
10	Validator → SSE → Browser	The answer is checked (prices allowed for this user) and streamed: *"DRV-03 re-sized from R7 to R8 at 88 % load. BOM: R7 removed, R8 added. Price updated."* An `item.updated` event makes the browser load the R8 model from Blob storage (lowest detail first) and refresh the BOM and price panels.
11	Worker (interactive) → Host project API	The worker claims the outbox row and saves the change to the host with `If-Match` and an idempotency key. The host's webhook for this change carries version 8, which is already applied, so it is ignored.
12	Audit log	The question, the preview and resize tool calls, the confirmation and the answer are stored.

What changes in other cases:

- **Sizing tool down or circuit open:** step 4 stops. The assistant says sizing is unavailable and nothing changes.
- **The user declines:** step 7 never runs. No write, no outbox row.
- **Same variant recommended:** step 8 writes nothing to the item or BOM; the answer says no re-size is needed.
- **Channel partner:** the flow is the same, but step 9 returns `not_permitted` without calling pricing, and the validator ensures the answer contains no price.

2. Technology choices

The guiding rule is **fewer moving parts**. The real bottlenecks are the external systems' rate limits and outages, not our own compute, so every component must earn its place. The reference stack (Node.js, Express, MongoDB, Redis) was the starting point; we kept what fits and changed the database, with the reasons below.

2.1 Choice per component

Component	Choice	Why
Runtime	Node.js 20 LTS, TypeScript (strict)	The service mostly waits on slow external systems. Non-blocking I/O handles many parallel calls per sidebar request cheaply. Strict types make the panel and adapter contracts explicit.
HTTP framework	Express	Small, well known and easy to review. Handlers stay thin; the logic lives in services, so the framework is not a lock-in.
System of record	PostgreSQL (Azure Database for PostgreSQL, Flexible Server)	Relational data and multi-row transactions. See 2.2.
Cache, rate limits, job queue, pub/sub	Redis (Azure Cache for Redis) with BullMQ	One component covers four needs. Rate limits apply to the whole service, so the counters must be shared by all API replicas, not held in process memory. BullMQ adds priority queues, job-ID dedupe, retries and a dead-letter queue. Pub/sub routes a push event to the replica that holds the user's connection.
Push to browser	Server-Sent Events (SSE)	Push is one-way (server to browser). SSE runs over plain HTTPS through the WAF, reuses the Bearer token and reconnects automatically.
Validation	zod	Validates configuration, request bodies and every vendor response. A schema failure is how a vendor API change is caught before users notice.
Tokens	jose (JWT and JWKS); MSAL for the OBO exchange	Standards-based checks of signature, issuer, audience and expiry. The identity provider supports the on-behalf-of flow for systems that need the user's own token.
3D models	Azure Blob Storage behind a CDN	glTF files of 2 to 40 MB must not flow through the API. Content-hashed paths are immutable and cacheable; the browser downloads them with short-lived SAS URLs.
Hosting and edge	Azure Container Apps in a private VNet, behind the customer's WAF	Containers in the customer's tenant, as required. Autoscaling by HTTP load and queue depth, internal ingress for workers, managed identity for every component.
Secrets	Azure Key Vault with managed identity	No secrets in code, images or environment files.
Logs and observability	pino (JSON logs), OpenTelemetry to Azure Monitor	Structured logs with request IDs, one trace per user action across adapters, and alerts on breaker state, rate-limit rejections and schema failures.
Tests and local setup	Vitest, supertest, Docker Compose	Fast HTTP-level tests against the real app. Postgres and Redis start locally with one command.

2.2 Why PostgreSQL instead of MongoDB

MongoDB is the reference stack's database, so this choice needs the clearest reasoning. It comes from the shape of the data and the consistency the workspace needs.

Need	PostgreSQL	MongoDB
Re-size must stay consistent. Variant, 3D model reference, BOM line, version and outbox row change together.	One ACID transaction across several tables. All of it is saved or none of it is.	Either embed the whole project in one document, which grows large and conflicts when several items are edited, or use multi-document transactions, which need a replica set and are not the usual Mongo pattern.
The data is relational. Project → item → variant → BOM line → price snapshot.	Foreign keys and unique constraints (one BOM line per item) are enforced by the database.	References between collections are checked only by application code.
Idempotency and concurrent edits.	`INSERT … ON CONFLICT` for upserts by revision, unique idempotency keys, and a version column for optimistic locking (409 on mismatch).	Possible with unique indexes and conditional updates, but each case must be hand-built.
Outbox to the host.	The outbox row is written in the same transaction as the change, which is what makes the pattern reliable.	Needs a multi-document transaction or change streams for the same guarantee.
Search over 12,000 variants.	Built-in full-text search plus trigram indexes on variant codes. No separate search engine.	Text indexes work; Atlas Search is a separate service outside the customer's tenant.
Variable product attributes.	`JSONB` columns give document-style flexibility where it helps, while the core relations stay strict.	Flexible documents are Mongo's strength.
Hosting in the customer's Azure tenant, with a security review.	First-party managed service with private endpoint, Entra ID login, zone-redundant HA and point-in-time restore.	Cosmos DB's Mongo API has compatibility gaps; Atlas runs outside the customer's subscription.
Scale (600 concurrent sessions, 5,000 users).	A light load. Read replicas and connection pooling cover growth to 50,000 users.	Also sufficient; scale does not decide this choice.

What we give up: the team already knows MongoDB, and its flexible documents would suit catalogue attributes and raw vendor payloads. `JSONB` covers that need without giving up transactions and constraints, which the re-size flow depends on.

2.3 PostgreSQL indexes

Indexes are chosen from the queries the service actually runs, not added to every column. Each one speeds up a hot read path or enforces a rule the data depends on. Three principles:

- **Index what the request path reads.** The sidebar has a 1 s budget, so its lookups (item by ID, catalogue by variant, price by variant + country + group) must be single index lookups.
- **Let unique indexes enforce rules.** One BOM line per item, one inbox entry per webhook event, one result per idempotency key. The database rejects a duplicate even if application code has a bug.
- **Keep write-heavy tables lean.** Every index slows down inserts. The outbox and audit log get only what their readers need, using partial and BRIN indexes to stay small.

PostgreSQL creates an index for every primary key and unique constraint automatically, but **not** for foreign keys, so those are added explicitly where they are queried.

Table	Index	Type	Why (the query or rule it serves)
items	`item_id` (primary key)	B-tree	Every sidebar, re-size and assistant call loads one item by ID. A single lookup of about a millisecond.
items	`project_id`	B-tree	Foreign key. Loading a project's layout, building its BOM and checking project membership all filter by project.
bom_lines	`item_id` UNIQUE	B-tree	Enforces exactly one BOM line per placed item, and lets the re-size transaction find that line directly.
bom_lines	`project_id`	B-tree	Foreign key. The project BOM endpoint reads all lines of a project.
price_snapshots	(`variant_code`, `country`, `customer_group`) primary key	B-tree	The step ② price lookup after a Redis miss. The column order matches the Redis cache key, so it is one exact lookup.
price_snapshots	`valid_until`	B-tree	The scheduled worker finds quotes about to expire for prefetch, and expired rows for clean-up.

sizing_results	(input_hash, api_version) UNIQUE	B-tree	The step ② sizing lookup. The same inputs on the same API version give the same result, so it is stored once and reused by every user.
catalogue_variants	variant_code (primary key)	B-tree	The catalogue panel reads one variant by code; items and BOM lines reference it.
catalogue_variants	search vector over name and description	GIN (full-text)	Word search across 12,000 variants ("IP55 drive 400 V") without a separate search engine.
catalogue_variants	variant_code with trigram operators	GIN (pg_trgm)	Partial and fuzzy code search, for example "R7" or "ACS880", which a normal B-tree cannot answer.
catalogue_variants	(frame, voltage, power_kw)	B-tree	Filters for search facets and for finding the variants that fit a sizing result.
catalogue_variants	attributes (JSONB)	GIN	Lets queries filter on variable technical attributes (options, IP rating) stored as JSON.
outbox	created_at WHERE status = 'pending'	Partial B-tree	The worker claims the oldest pending rows. The index holds only unsent rows, so it stays tiny however large the table grows.
webhook_inbox	event_id UNIQUE	B-tree	Deduplicates webhooks: a repeated event from the host fails the insert and is processed only once.
idempotency_keys	key UNIQUE	B-tree	A repeated re-size click with the same key returns the stored first result instead of applying twice.
audit_log	created_at	BRIN	Append-only and time-ordered. A BRIN index is a fraction of a B-tree's size and fits time-range queries.
audit_log	(user_id, created_at)	B-tree	Answers "what did this user (or the assistant on their behalf) do" during reviews and security investigations.
assistant_memory	(conversation_id, created_at)	B-tree	The orchestrator loads the recent turns of one conversation in order.

Verified, not guessed: the hot queries are checked with `EXPLAIN ANALYZE` in tests against seeded data, and unused indexes are found with `pg_stat_user_indexes` and removed.

2.4 Other alternatives considered and rejected

Alternative	Why not
Microservice per enterprise system	More network hops inside a 1 s budget, cross-service coordination for global rate limits and a larger attack surface. Splitting would not raise the external limits. Adapters stay isolated, so any one can be extracted later.
Kafka or Azure Service Bus	Redis with BullMQ already covers priority, dedupe and retries with one fewer component. Service Bus is the upgrade path if stronger delivery guarantees are needed.
Elasticsearch or Azure AI Search	12,000 variants fit comfortably in Postgres full-text and trigram search. Another cluster adds cost and operations.
WebSockets	Push is one-way; SSE is simpler and passes the WAF as plain HTTPS.
GraphQL gateway	Adds a schema layer, and partial failures are less explicit than a status per sidebar section.
API gateway product as the backend	It can route and throttle, but cannot hold the sidebar aggregation, per-section statuses or entitlement filtering. At most it sits in front as the edge.

3. Key sequences

All three sequences use the same project: a water-plant pump room with drive DRV-03, currently variant `ACS-R7-75-400` (frame R7, 75 kW motor) at item version 7. Every data read follows the same order: ① Redis, ② PostgreSQL, ③ adapter to the external system only on a miss, ④ write the result back.

3.1 Sequence 1: the engineer clicks a drive and the sidebar fills (sizing tool down)

Goal: useful content in every panel within 1 s, even though the sizing tool is down.

#	Component	What happens
1	Browser	The engineer clicks DRV-03. The 3D view highlights it immediately on the client; no network call is needed for that.
2	Browser → WAF → Integration API	`GET /api/items/DRV-03/sidebar` with the Bearer token.

3	Integration API	Verifies the token, builds the caller context (`sales_engineer`, DE, group G1), checks that the user is a member of this project, and loads the item from PostgreSQL.
4	Integration API: fan-out	Starts the four panels **in parallel**, each with its own deadline of about 800 ms.
4a	Catalogue panel	① Redis hit for `ACS-R7-75-400` → ok. (On a miss, ② the catalogue read model in PostgreSQL answers; the catalogue system itself is never called on this path.)
4b	Sizing panel	① Redis: miss. ② PostgreSQL holds DRV-03's last sizing result (R7, 72 % load). It needs a refresh, but the **circuit breaker is open** because the sizing tool is failing, so ③ the sizing tool is **not called**. The stored result is returned as `stale` with "sizing unavailable since 13:40". If no stored result existed, the status would be `unavailable`.
4c	Pricing panel	The entitlement check passes (sales engineers may see prices). ① Redis hit for R7 / DE / G1 → ok. On a miss: ② a still-valid PostgreSQL price snapshot, else ③ the ERP within its rate limit. If the ERP cannot answer before the deadline, the panel returns `pending` and a job finishes the call.
4d	Documents panel	The document list for the variant comes from Redis or the document vault (with the user's OBO token) and is filtered by this user's permissions → ok.
5	Integration API → Browser	Responds within about 900 ms with one section per panel: catalogue ok, sizing `stale`, pricing ok, documents ok. Each panel renders on its own; the sizing panel shows its banner.
6	Redis Pub/Sub → SSE	Any `pending` section is completed by a worker and pushed as a `panel.updated` event. Redis Pub/Sub routes it to the API replica that holds this user's SSE connection.

Result: one failing system turns one panel `stale`; the other panels are unaffected and the response stays inside the 1 s budget. During the nightly sizing maintenance window the breaker is opened in advance, so users see the same behaviour without waiting for failures.

3.2 Sequence 2: the engineer raises the motor load and the drive is re-sized

Goal: the drive moves to a larger frame, the 3D model swaps, and the BOM and price update, all consistently.

#	Component	What happens
1	Browser → WAF → Integration API	The engineer raises the motor from 75 kW to 90 kW in the sizing panel. `POST /api/items/DRV-03/resize` with the new motor data, an `Idempotency-Key` and `If-Match: 7`.
2	Integration API	Verifies the token, checks that the user may edit this project, and checks the idempotency key (a repeated click returns the first result instead of applying twice). If the sizing breaker is open, it returns `503 "sizing unavailable"` and nothing changes.
3	Integration API: sizing read order	① Redis `size:{hash(inputs, apiVersion)}`: miss. ② PostgreSQL sizing results for that hash: miss. ③ The sizing adapter queues an interactive job whose ID is the input hash, so identical requests merge.
4	Worker (interactive) → Sizing tool	The worker takes a token from the sizing rate limit in Redis, uses the user's OBO token and calls the sizing tool (1 to 6 s). The response is validated against its schema: recommended `ACS-R8-90-400` (frame R8), 88 % load. ④ The result is stored in PostgreSQL and Redis.
5	Integration API	The API waits a short time (about 1.5 s) for the job. If the result is ready, it continues in the same request. Otherwise it returns `202 pending` and the next steps finish on the server, with the outcome pushed over SSE.
6	PostgreSQL (one transaction)	R8 differs from R7, so one transaction updates the item's variant and 3D model reference, its BOM line, sets version 8 and writes an outbox row. All of it is saved or none of it is. The `item_id` stays DRV-03.
7	Pricing	The entitlement check passes. Price for R8 in DE / G1 follows the read order (Redis, PostgreSQL snapshot, ERP) → ok or `pending`.
8	Integration API → Browser	The response (or an `item.updated` SSE event) carries the new variant, signed URLs for the R8 model, the BOM change (R7 removed, R8 added) and the price difference.
9	Browser → Blob storage	The browser loads the R8 glTF model through the CDN, lowest detail first, and swaps it into the scene. It re-runs the layout rules: a larger frame may now break a clearance rule and turn red. The BOM and price panels update.
10	Worker (interactive) → Host project API	The worker claims the outbox row and saves the change to the host with `If-Match` and an idempotency key. On a conflict it reloads from the host and tells the user. The host's webhook for this change carries version 8, already applied, so it is ignored.

If the variant does not change: only the sizing result is stored; no item, BOM or host write happens and the response says `changed: false`.

3.3 Sequence 3: "What will this layout cost, and which item has the longest lead time?"

Goal: a correct answer for a user who may see prices, and a safe answer for one who may not, from the same flow.

#	Component	What happens
1	Browser → WAF → Integration API	`POST /api/assistant/messages` with the question and the Bearer token. The API verifies the token and forwards the question and caller context to the Assistant container.
2	Orchestrator	Routes the question to the BOM agent and loads the conversation memory from PostgreSQL.
3	BOM agent + Private LLM	The model chooses two read tools: `get_bom_cost(projectId)` and `get_lead_times(projectId)`. Reads need no confirmation.
4	Tool registry → Integration API	Both tools call the same project and pricing services as the sidebar, as this user, through the same entitlement check. Prices come mostly from Redis and PostgreSQL snapshots, because the scheduled worker prefetched them when the project was opened; misses go to the ERP at low priority within its rate limit.

From step 5 the two users differ:

#	Sales engineer (may see prices)	Channel partner (may not see prices)
5	Tools return every BOM line with price (DE, group G1), totals, quote validity and lead times.	The entitlement check denies prices before any cache or ERP call. Tools return the items with `pricing: "not_permitted"` and no price or lead-time fields at all. The model never receives a price, so no prompt can extract one.
6	The model drafts: "*Estimated total €184,300 (prices valid until 22 Oct 14:02). Longest lead time: DRV-03, frame R8, 12 weeks.*" Any stale or pending price is stated as such, never guessed.	The model drafts: "*Pricing and lead times are not available for your role. The layout has 6 items; your sales contact can provide a quote.*"
7	The validator confirms that every figure comes from a tool result and that this user may see prices.	The validator confirms the answer contains no price, currency amount or lead time. If it did, the answer would go back to the agent with a critique.
8	The answer is streamed to the browser over SSE. The question, both tool calls, the entitlement decisions (including denials) and the answer are written to the audit log.	

Note: lead time is treated as commercially sensitive and follows price visibility. This is an assumption and a per-field policy setting, so it can be changed without code.

4. Assumptions

1. **Identity:** Microsoft Entra ID is the identity provider. It authenticates users and issues the claims used for authorisation (role, country and customer group). The Integration API makes every authorisation decision from those claims, on the server.
2. **Rate limits:** the vendors' rate limits (sizing 20 requests per second, pricing 5 requests per second) apply to our whole service, not to each user. All API replicas and workers therefore share one set of counters in Redis.
3. **Cache lifetimes:** a sizing result depends only on its inputs and the sizing API version, so it is cached and reused for 7 days. A price is cached for as long as its quote is valid (24 hours) and is never served after the quote expires.
4. **Stale data:** when an external system is down or unreachable, users prefer to see the last known data, clearly marked as stale with its age, rather than an empty panel.
5. **Re-sizing:** a re-size updates the existing item's variant and its stored sizing result. The item keeps its identity (`item_id`); it is never replaced by a new item.